COMP101
Introduction to Programming
2025-26

Lab-01 Week-01
Drop-In – no supervision

Workbook-02
Integrated Development and Learning Environment
IDLE

THE IDLE EDITOR IS WHERE WE PROGRAMME

IDLE editor
(Integrated Development and Learning Environment)
String output, save/run, error messaging, retrieving a Python file

We programme in the IDLE EDITOR - don't program using the shell (see workbook-03).
By all of us using the IDLE editor, support and compatibility with the test machines should ensure trouble-free programming.

When reading around or using examples on the web etc. check they are **not written in Python v2.x especially** - you may find they don't execute properly and will need amending due to compatibility issues.

Typing in code ('coding') from an algorithm (a defined sequence of steps, usually in pseudocode) to program a computer can be tedious and error-prone. All high-level languages provide some sort of support to make the task easier and more efficient. This support is usually in the form of a customised text editor with integrated features to make coding easier.

Python's integrated environment is called IDLE.
It is a text editor to help write/edit and debug Python scripts (lines of code ready to be executed as a program) and provides extra functionality of syntax highlighting (colours etc), smart indentation and no feature clutter to present a usable interface for coding.
It has shown to be very good for beginners, especially within the academic environment, allowing users to focus on their programming rather than the environment.

If you want the Python IDLE editor, on the drop-down menu click IDLE (Python 3.7 64-bit). A slightly different version of the shell appears – it looks like below (there may be some minor variation on this depending on the version of Python – but it doesn't matter)

Ex1
IDLE screen editor – to type in a program and save it as a .py file

Python opens in the shell window (see workbook-03) - we need to get out of this so we can start to program.
We program in the IDLE editor ('script editor') and the results appear in the 'shell window' on successful execution.

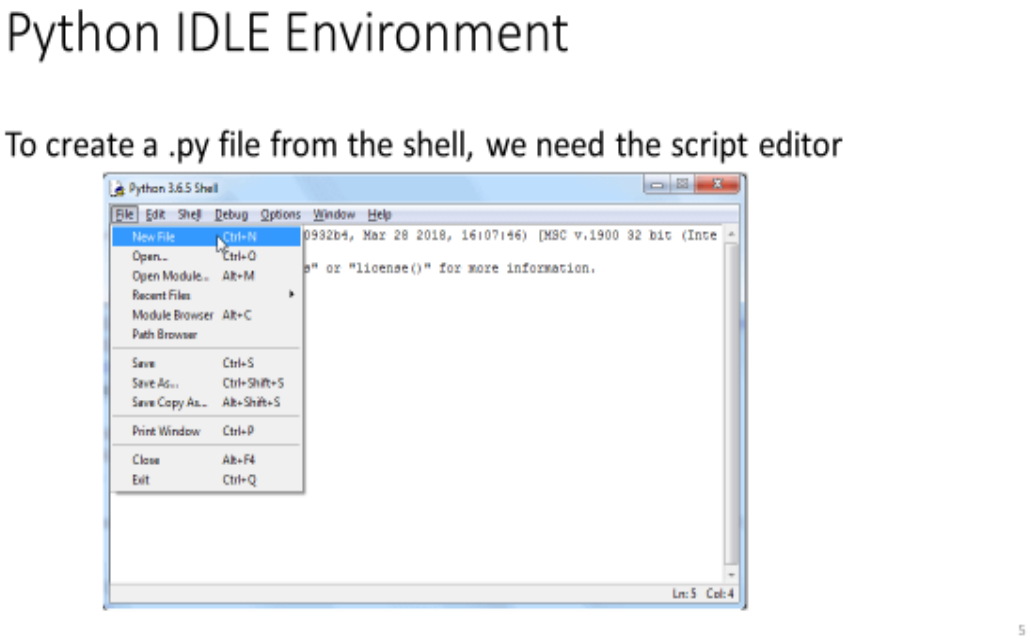


Figure-01

The first thing we see is the 'shell window'. We need to open IDLE to begin to program.
Click on File → New File to access the script editor where we can type in our program statements.

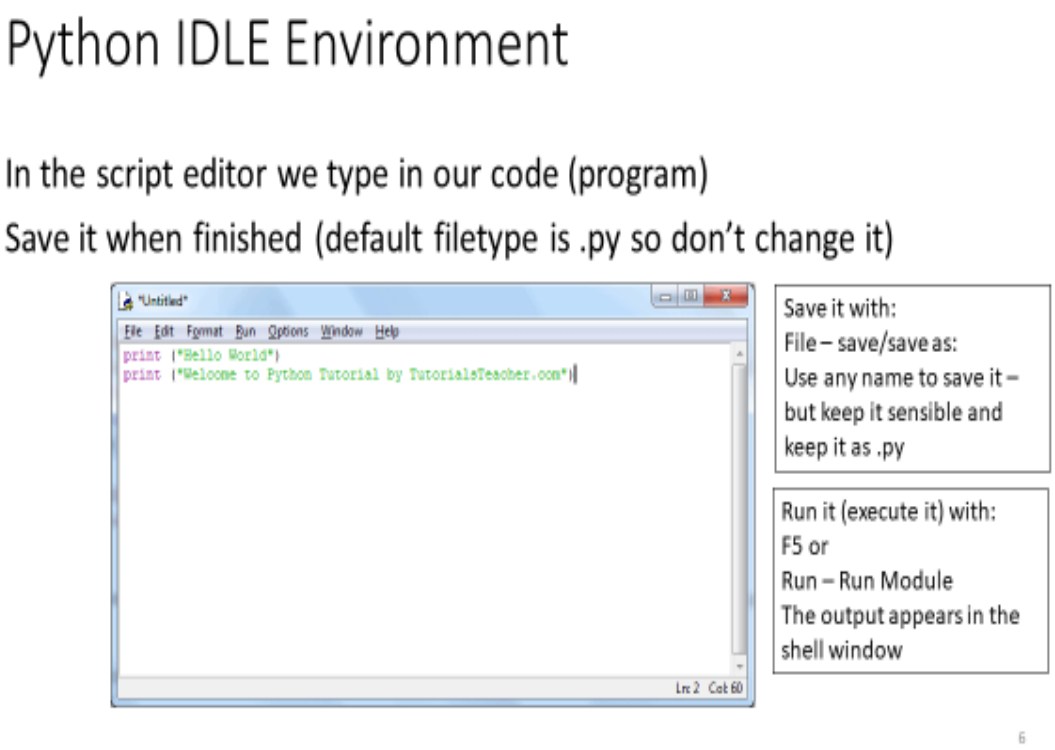


Figure-02
After we access IDLE from the shell window, we can type in our code. The above is an example of two statements that make up a single Python program.

When complete, we must save the code (as a .py file – the .py extension is allocated automatically – there is no need to type it in as part of the filename).

When we run ('execute') the saved program, the output appears in the shell window:

Python IDLE Environment

On execution of a Python script as a .py file output appears in shell

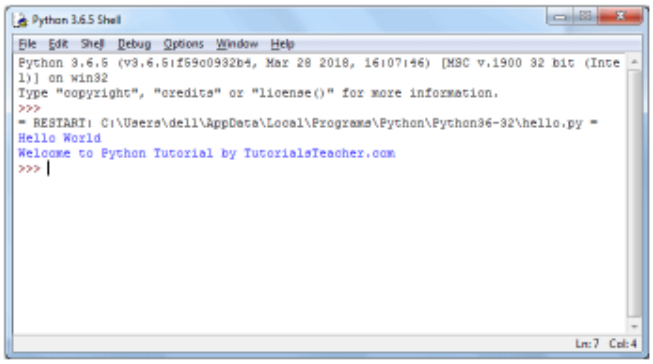


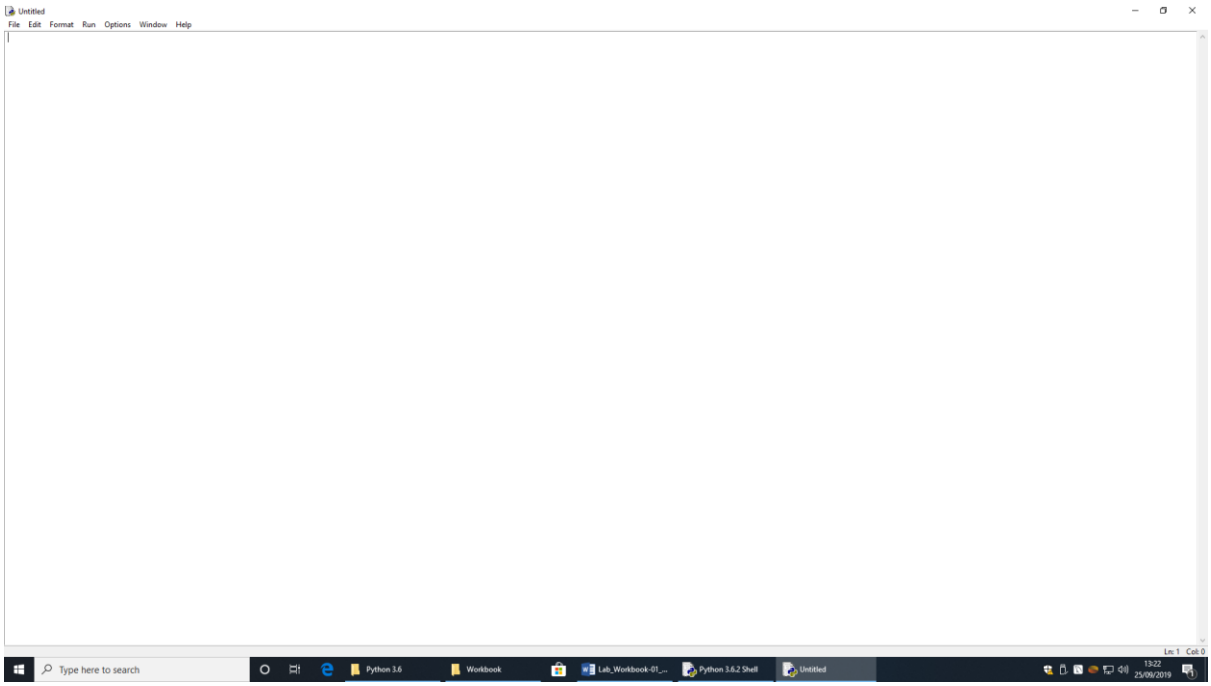
Figure-03

Close the shell window and we are returned to the script editor

Ex2
Open the IDLE editor from the shell window

In the command shell, click:
File → New File

This opens the IDLE editor screen – it looks like this (may be some variation)



This is where we type our Python code. The top line tells you the name of the program – at the moment it is 'untitled'.

Programming via IDLE is a 3-stage process:

1) Type your code in
(try the previous two lines of code from TutorialsTeacher.com)

2) Save it as a .py file
File → Save or Save As …
(the .py extension is done automatically)

3) Run it (i.e. 'execute' it)
Run → Run Module or press function key F5 on the keyboard
If you have not saved it before you run it, you will be prompted to save it

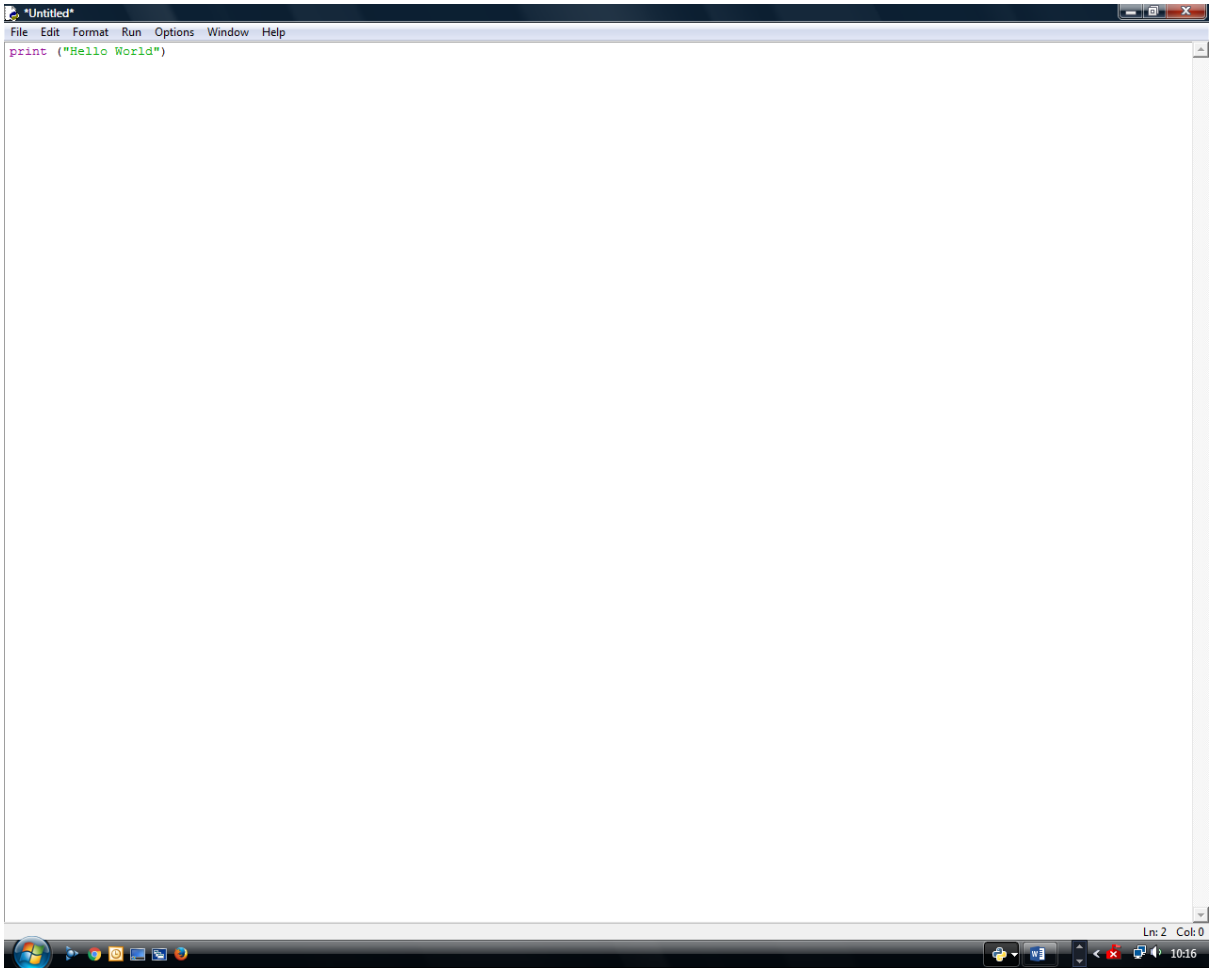
Results appear in the shell window. You can close it and return to IDLE.

ex3
Python code to output a string

Make sure you are in the IDLE editor and not in the shell
(i)
Type the following exactly as you see it here – don't forget the brackets and quotation marks:

```
print ("Hello World")
```

It should look like the screen shot below:



NOTE:
In other languages, it is common to find a character is used to denote the end of a line (it may be a slash character, a semi-colon character etc).
Python does not use this format – simply press return at the end of the line you have typed and carry on coding. Pressing return at the end of line indicates to Python the line of coding is complete and a new line of code follows it.

IDLE ‘takes charge’ of the formatting in this screen to help the programming process– take a look at the different colours – it can come in useful when we have to debug larger programmes.

After you type the code in, nothing happens – yet.
If this was the command shell, the words Hello World would appear after the >>> prompt when we press return.

Ex4

Save and run:

To execute our code, we must **save it first as a .py file** and then run ('execute') the file. We can only execute saved files.

If we click run before saving, Python will prompt us to save it with the message 'Source code must be saved'.

Invent a consistent naming system for your .py files so you can recognise them for amendments/cutting and pasting etc.

(i)
Using your Hello World program:

- Click File – Save (or Save As…)

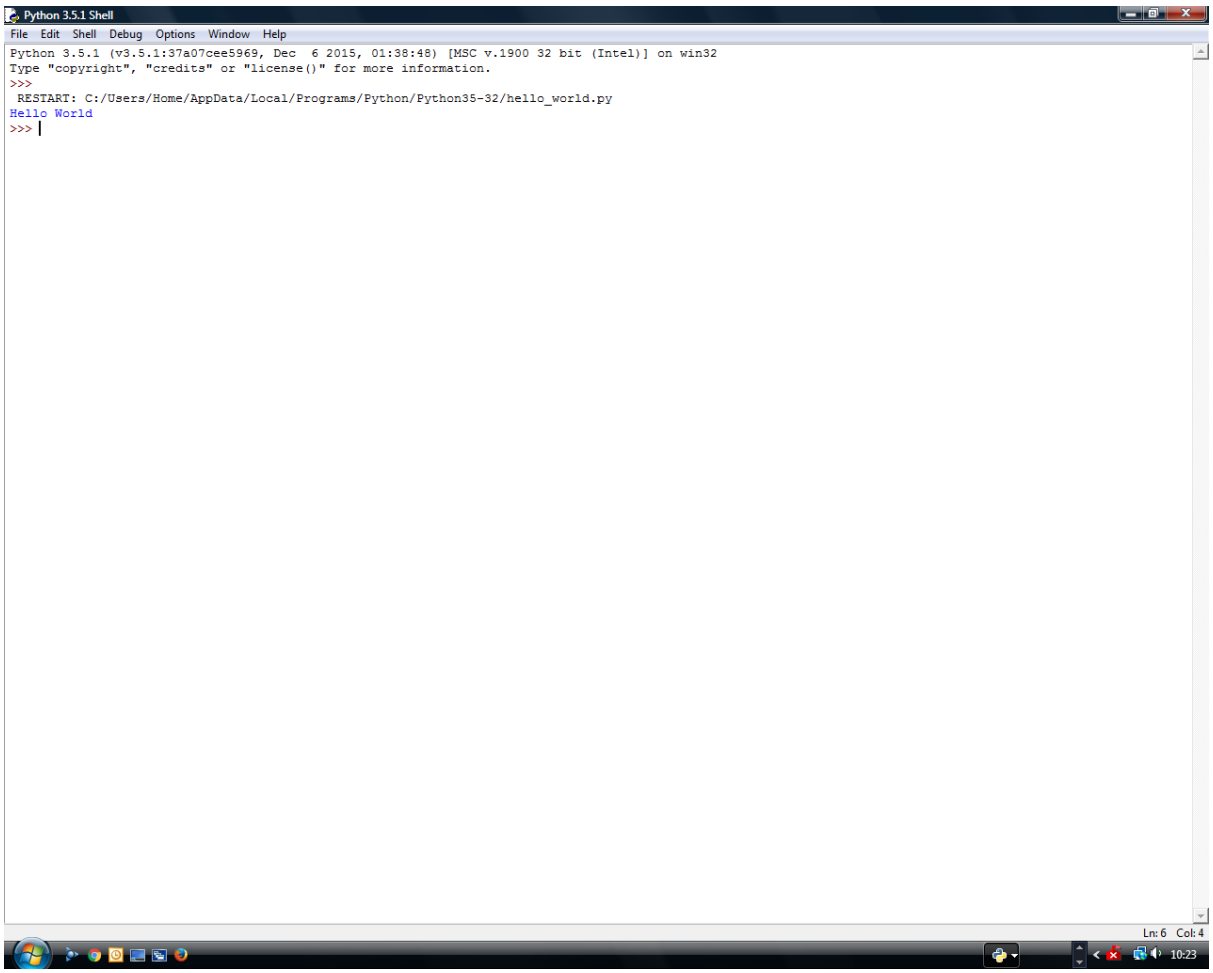
- Give it a name and remember where you saved it to (use sensible file names)
  The file will be saved automatically with a .py extension – don't change this.

- Click Run
  The command shell screen appears and the words Hello World are output in the command shell screen

Summary of creating and executing a .py file:

1. Python always opens in the shell window
2. To create a new Python program click 'File' → 'New File'
3. IDLE editor opens – type in your code
4. Save your code as a .py file
5. click 'Run'
6. Output opens in the shell window

It should look like the screen shot below:
This is the shell window.
Output is always left-justified to the screen unless we tell it something different:



You can close the shell screen to return to IDLE

Sometimes, when you run (‘execute’) your program. It may look like the screen ‘flashes’ and then nothing seems to have been output.
Your program has worked, but the output has been shown and the screen refreshes ready for the next operation.

If this happens, you can ‘hold’ the screen until you have read it.
Simply amend your code to look like the following:

It has a line in red – this is a ‘comment’ and is ignored by the interpreter

```
#extra input() statement holds the screen until return is pressed by the user
print ("Hello World")
input()
```

ex5
Error Messaging

(i)
With your Hello World program, try forcing some errors just to get a 'feel' of what they look like:
Save these as the same filename, overwriting the original, or as separate files – it doesn't matter.

Try:
print (Hello "World") #mis-use of quotes

print ("Hello World" #mis-use of parentheses

print ('Hello World") #mis-use of mixed quotes

ii) Try your own on the statement just to see what error messages look like.

The error messages may not make sense at first, but recognising them in general terms can help you identify the issue they are flagging and will help later with larger programmes.

Remember Python is case sensitive.

Ex6
Re-loading existing files and amending them

(i)
Close Python and then re-open it

(ii)
Open up the IDLE screen and re-load your 'Hello World' programme – remember you are looking for a file with extension of .py

(iii)
Click on File – Open … and select your Hello World file

(iv)
Practice controlling the output:
Try the following:

- Output the string "Hello World" (with 5 spaces between the words).
  Save and run it.

- Output the string "Hello World" (with 3 tabs between the words)
  Save and run it.

Remember if you save a file as an existing name it will overwrite the original file – it doesn't matter here.

Ex7
Coding multiple lines
(i)
Output a menu as shown below:
(The blue colour is not relevant – used here for illustration only)

Main Menu

D: Division
M: Multiplication
A: Addition
S: Subtraction

- You will need six lines of code using the 'print' command, one for each line in the menu.
- The menu heading is underlined
- There are spaces between the menu options and the menu descriptors
- The menu will appear left-justified to the output screen – that's okay

Save and execute is as normal and see what you get, debugging as needed.

Ex8

Comments - # symbol

Comments can be useful to explain any tricky bits of code. They are also useful in debugging to help understand where a problem may be by isolating specific lines from execution.

(hash symbol)
A single '#' at the start of any line will make the entire line go red and the line will not be executed

Try the following using your Main Menu program
(The comment lines here are just to help you identify which line you need to put a hash symbol in front of at the start)

```
print ("Main Menu")              #line-01
print ('--------------')         #line-02
print ("D Division")             #line-03
print ("M Multiplication")       #line-04
print ("A Addition")             #line-05
print ("S Subtraction")          #line-06
```

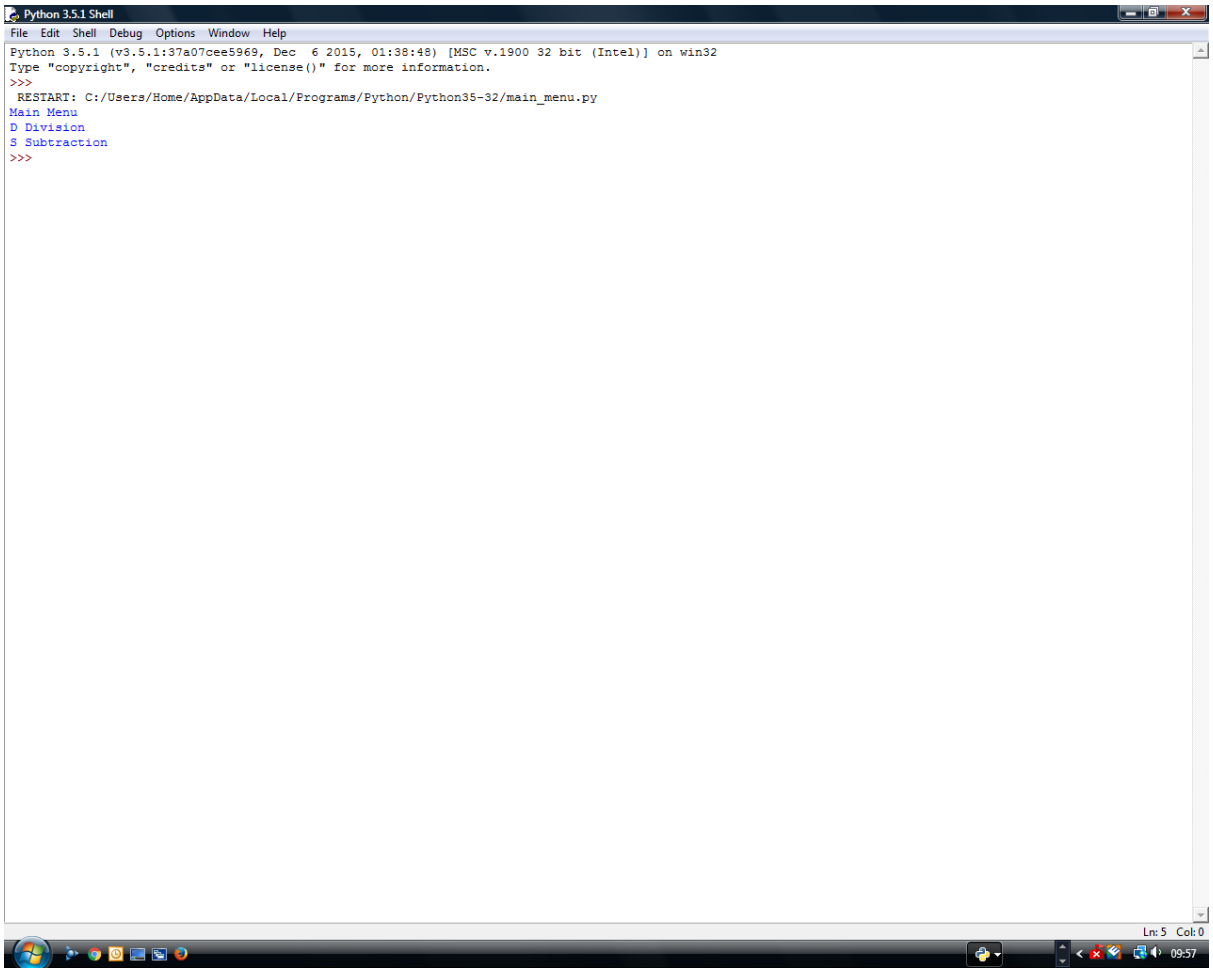
(i)
Comment-out lines 2, 4 and 5 - by putting a '#' as the first character in the line.
The commented lines go red:

- Line-2 – the underline string
- Line-4 – the option M string
- Line-5 – the option A string

See what output you get:

(ii)
Save it (best to use a different name) and run the file – you should see that the underline for the heading is not output and options M and A do not output, i.e. they are ignored by the interpreter.

See the screen shot below:



Ex9

Output blank lines with the print( ) statement

A blank line is output with the statement:

```
print ( )       #common error is to miss the parentheses
                #there does not have to be a space in the parentheses
```

(i)
Amend your main menu program to prompt the user to select an option.
The prompt is to be separated from the menu by a blank line and is to be in the format:

Enter an option:

There should be a space after the colon. This is cosmetic to make it easier for the user.
e.g. if the user selects option-A, they should see:

Enter an option: A

There will be a space between the colon and letter A

You don't need to do anything else, i.e. the program does not have to execute the option selected – it should stop with an error message.

```
Main Menu
---------------
D:    Division
M:    Multiplication
A:    Addition
S:    Subtraction

Enter an option:
```

There is a blank line here

Ex10
Output using triple quotes for ‘multiple line spanning’

Outputting a lot of text in specific positions so it lines up can be time-consuming and often leads to guesswork about the spacing etc.

(i)
Try the following code as shown – the advantage is that, as a programmer, you can see the alignment taking shape within the editor screen:

Here the menu will all left-align on itself, two tabs in from the left-hand of the screen

```
print('''                #use of triple quotes to open
\t\tMain Menu          #\t outputs a single 4-space tab – here there are two of them
\t\tD: Division
\t\tM: Multiplication
\t\tA: Addition
\t\tS: Subtraction
print()
\t\tEnter an option:
''')                    #use of triple quotes to close
```

This is called ‘multiple line spanning’

This can be a tidier (and easier?) way to output columnar/tabular strings as the output reflects the relative physical position of the code itself, i.e. we can see the alignment of the text whilst we are coding it

Ex11
Execute a program

Load and execute lab01_sequence.py

Study the output in the output window against the code window to see what is happening in a sequential program.

The over-commenting that is present in the code has been left out here so you can see the 'active code', i.e. the code that really matters.

```
print ('Main Menu')
print ('---------')
print ('D: Division')
print ('M: Multiplication')
print ('A: Addition')
print ('S: Subtraction')
print ('X: Exit Menu: ')
print()
print ('Enter an option D,M,A,S or X to exit: ')
```

Have a look at:
1) The colours in the editor – makes life easier when reading the code

2) The print() statement is used to output a message to user.
It expects no input so when the line outputs its message, the next line in sequence executes

3) The input() statement is also used to output a message to the user.
It expects input so execution stops until the user inputs something.
We can use the message to tell the user what to do so execution continues to the next line in the sequence.
If we don't put a message into the input() statement, the program will still stop executing at that point but the user is not being told what to do to get it going again.

4) print() – to output a blank line

5) The use of 'whitespace' between the lines of code to aid readability

6) The use of commentary (in red with # in front).

Don’t over-comment. This practice code is over-commented deliberately – normally use commentary to explain any tricky or unusual coding

Ex12
Debug someone else’s program

(i)

Load and execute lab01_sequence_bugged.py.

It won’t work as there are bugs in it.

Debug the code to make it work.

Have a look at:

- The error messages – at first they don’t make sense but when you see a lot of them you’ll get the idea of what they really mean
- The error may not be where the interpreter has stopped – it may be on the line above or may be further back in the line where the error is. This is because the code is ‘read-ahead’ before it executes. The code read by the interpreter may have a bug in it but by the time we get the error message, the next part of the code has been read in advance and the program stops – which makes us think the error is at the point the code has stopped when in fact it is before it.
- Once you get it working, are you SURE you have spotted ALL the errors? Check the output.
- HINT: What happens if we don’t press Return to exit the program?
